INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: STUDENT PERFORMANCE AND STUDY-HOURS DASHBOARD

NAME	CLASS ROLL NO.	ENROLLMENT NUMBER
Sohana Nasrin	119	12025052003128
Koyel Hazra	115	12025052003124
Anusha Pattanayak	122	12025052003131
Anusuya Dalapati	117	12025052003126
Sushmili Nag	113	12025052003121

SECTION: B

SUBMITTED TO: Prof. Indranil Basu

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026–2027

DATE OF SUBMISSION: 6 November 2026

Abstract

This project presents a Python-based Student Performance and Study-Hours Dashboard for exploring the relationship between reported study time, subject assessment scores, and attendance. The application uses a synthetic CSV dataset of 20 student records and combines Python functions, an object-oriented StudentRecord class, NumPy, Pandas, Matplotlib, and Streamlit. It validates and analyses records, derives average performance measures, compares Python, Data Structures and Algorithms (DSA), and Mathematics scores, identifies potential outliers using an interquartile-range approach, and presents visual summaries. The dataset is fictional and is used only to demonstrate software behaviour. In this dataset, the mean reported study time is 4.33 hours and the mean student average score is 68.28/100. The Pearson correlation between study hours and average score is 0.821. These descriptive findings do not establish that study time causes higher marks. The project demonstrates a reproducible, software-only workflow for data handling, analysis, visualisation, and dashboard presentation.

Keywords: Python; Streamlit; student performance; study hours; Pandas; NumPy; Matplotlib; CSV; correlation; outlier analysis.

Table of Contents

1. 1. Introduction
2. 2. Problem Statement and Objectives
3. 3. Software and System Requirements
4. 4. System Design and Methodology
5. 5. Implementation of Major Modules
6. 6. Results and Discussion
7. 7. Testing and Validation
8. 8. Limitations and Future Scope
9. 9. Conclusion
10. 10. References
11. 11. Appendix: Dataset Schema and Execution

1. Introduction

Student performance records commonly include assessment marks, attendance, and estimates of time spent studying. When these values are stored in a CSV file, comparing subjects, identifying unusual values, and examining possible relationships can require repetitive manual work. A dashboard can make these operations more consistent and easier to interpret.

The Student Performance and Study-Hours Dashboard is a software-only Python project. It loads a local CSV dataset, calculates performance measures, performs statistical analysis, and presents the results through a Streamlit interface. Matplotlib visualisations support comparison of study hours, average scores, subject performance, score distributions, and potential outliers.

2. Problem Statement and Objectives

Problem statement: Develop a maintainable Python application that analyses synthetic student records and presents useful descriptive summaries of study hours, subject marks, and attendance, while validating inputs and avoiding unsupported causal conclusions.

- Load student records from a structured CSV file and validate relevant numeric fields.
- Calculate average scores and assign performance categories using the project's grade function.
- Represent individual student records through an object-oriented StudentRecord class.
- Use NumPy and Pandas for numerical summaries, subject comparisons, and data processing.
- Use the interquartile-range (IQR) method to flag potential score outliers.
- Generate Matplotlib charts and provide a Streamlit dashboard for exploring results.
- Export analytical summaries and chart files for review.
- Test core functions and document limitations of the synthetic dataset.

3. Software and System Requirements

Component	Purpose
Python 3	Core program logic and execution
Streamlit	Browser-based dashboard interface
Pandas	Tabular data loading, filtering, aggregation and export
NumPy	Numerical calculations and statistical operations
Matplotlib	Charts and visual summaries
CSV dataset	Input records containing student IDs, study hours, scores and attendance
VS Code / terminal	Development, execution and testing environment
Git / GitHub	Version control and source-code submission

The application runs on a computer with Python installed and the packages listed in requirements.txt. The dashboard is launched locally in a web browser using Streamlit; it is not a hardware-based project.

4. System Design and Methodology

4.1 Input data and schema

The supplied data/students.csv file contains 20 synthetic records. Its fields are student_id, study_hours, python_score, dsa_score, math_score, and attendance_pct. Scores and attendance are represented on a 0–100 scale; study_hours represents the reported study-time value stored for each record. The average score is derived from the three subject scores rather than stored as an independent input.

4.2 Data preparation and validation

The analysis workflow reads the CSV, checks required columns and numeric values, validates score and study-hour inputs, and derives performance metrics. Invalid or missing values should be handled according to the validation rules implemented in the source code. The example dataset is fictional and contains no real student personal records.

4.3 Analysis workflow

12. Load the student CSV dataset.
13. Validate the required fields and numeric ranges.

14. Calculate average scores and grade/performance categories.
15. Use StudentRecord objects to encapsulate record fields and related operations.
16. Compute descriptive statistics and compare subject averages.
17. Apply IQR-based detection to identify potential outlier scores.
18. Generate plots and export summary files.
19. Display key metrics, tables, and charts in the Streamlit dashboard.

4.4 Statistical measures

The mean summarises the arithmetic average and the median gives the middle value after sorting. Standard deviation describes spread. The interquartile range is IQR = Q3 − Q1; under the common 1.5 × IQR rule, values below Q1 − 1.5(IQR) or above Q3 + 1.5(IQR) are flagged. Pearson's correlation coefficient measures linear association between two numeric variables and ranges from −1 to +1 when defined. These measures describe this synthetic dataset and should not be interpreted as evidence of causation.

5. Implementation of Major Modules

Module / file	Role in the project
analysis.py	Data loading, validation, average/grade functions, descriptive and advanced analysis
student_record.py	StudentRecord class for structured record representation and related methods
visualizations.py	Matplotlib chart creation and saving of visualisations
main.py	Command-line integration of analysis, visualisations, and exported summaries
app.py	Streamlit dashboard with metrics, tables, charts, and interactive views
data/students.csv	Synthetic input dataset of 20 student records
test_*.py	Checks for setup, data, analysis, record handling, visualisations, and main workflow
requirements.txt	Lists the external Python packages required by the project
README.md	Project overview, setup guidance, features, and execution instructions

5.1 Dashboard and output design

The Streamlit application presents summary metrics and analytical sections for overview, student records, subject analysis, and outlier analysis. The underlying visualisation module generates a study-hours versus average-score scatter plot, subject comparison chart, average-score histogram, grade-distribution chart, and outlier visualisation. Exported outputs from the integrated workflow include student performance results, subject comparisons, potential outliers, and a text summary.

5.2 Average score and performance categories

For each record, average score is calculated from Python, DSA, and Mathematics marks. The project's get_grade function maps the average to a grade category. The precise thresholds are defined by the source code; this report does not substitute a different grading scheme.

6. Results and Discussion

The following values are recalculated from the 20-row synthetic dataset described in Section 4.1. Average score means the arithmetic mean of the three subject marks for each student. Standard deviation is reported as a population standard deviation for this summary table.

Indicator	Observed value
Number of synthetic records	20
Mean study hours	4.325 hours
Median study hours	4.00 hours
Mean of student average scores	68.28 / 100
Median of student average scores	70.67 / 100
Population standard deviation of average score	15.74
Minimum / maximum average score	35.67 / 94.33
Mean attendance	82.30%
Pearson correlation: study hours vs average score	0.821
Pearson correlation: attendance vs average score	0.959
IQR outliers in average score	0 (fences 24.38 to 113.38)

6.1 Subject-wise summaries

Subject	Mean score	Minimum	Maximum
Python	69.00	35	95
DSA	65.20	32	92
Mathematics	70.65	40	96

Among the three subjects, Mathematics has a mean score of 70.65, Python has 69.00, and DSA has 65.20. These are descriptive summaries of the constructed sample, not comparative measures of course difficulty or teaching effectiveness.

6.2 Relationship between study hours and average score

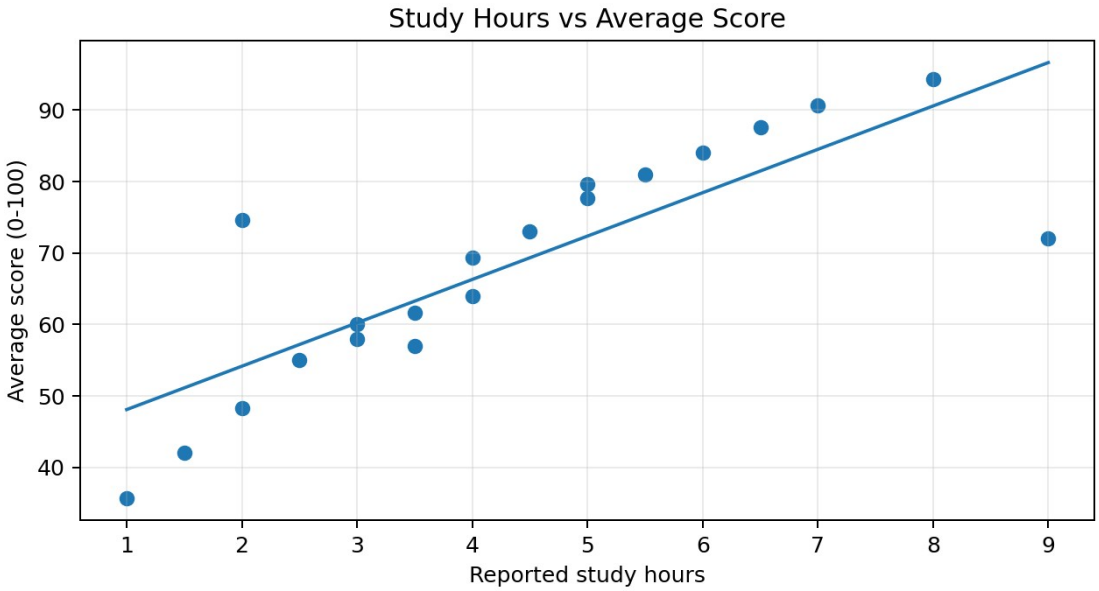


Figure 1. Reported study hours versus each student's mean of three subject scores.

The Pearson correlation in the supplied dataset is 0.821. This describes the direction and strength of a linear association in these 20 generated records. The sample is too small and synthetic to support general claims about students, and correlation does not establish causation.

6.3 Subject comparison and score distribution

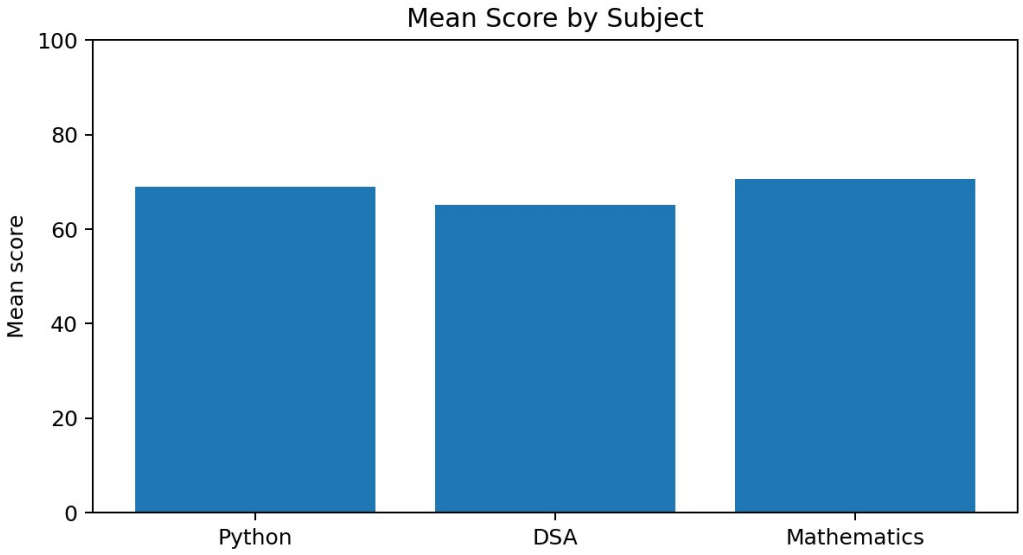


Figure 2. Mean scores for Python, DSA, and Mathematics.

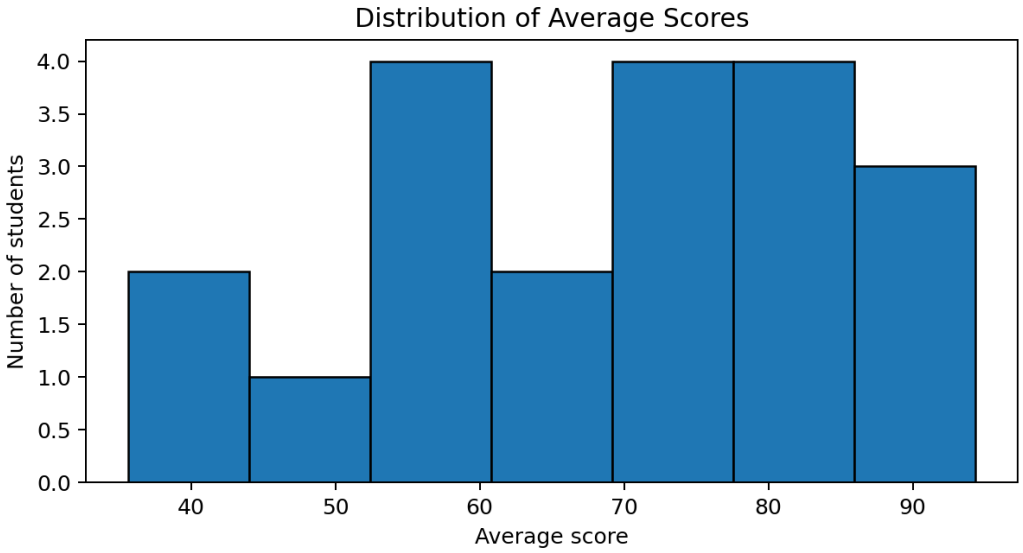


Figure 3. Distribution of student average scores.

6.4 Attendance and potential outliers

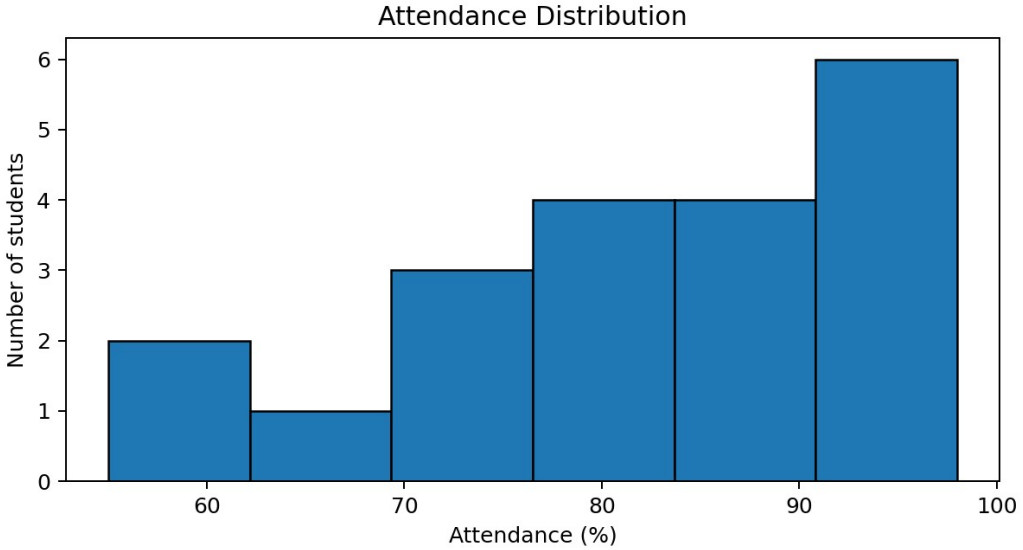


Figure 4. Distribution of attendance percentages in the synthetic dataset.

For student average scores, Q1 is 57.75, Q3 is 80.00, and the IQR is 22.25. Under the 1.5 × IQR rule, the lower and upper fences are 24.38 and 113.38; 0 records fall outside these score fences. An outlier flag indicates an unusual value under a statistical rule, not necessarily an erroneous record. The project's broader outlier visualisation may evaluate the variables specified in its code.

7. Testing and Validation

Test area	Expected behaviour / check
Environment setup	Required packages import and the Python environment runs the project
Dataset loading	CSV can be read and expected columns are present
Input validation	Out-of-range or non-numeric values are rejected or handled according to implemented validation
Average and grade functions	Derived average and performance category are consistent with the implemented rules
StudentRecord class	Record construction, validation, and methods return expected values
Advanced analysis	Statistics, subject comparisons, and outlier detection execute without errors
Visualisations	Chart functions create the expected image outputs
Main workflow	Integrated analysis and output-export process runs
Streamlit dashboard	Dashboard starts and its main views render from the local dataset

The repository contains separate test scripts for the main functional areas. A submission-time verification should run those scripts in the project's virtual environment and manually open the Streamlit dashboard. This report does not claim a specific test-pass count beyond the project status previously reported.

8. Limitations and Future Scope

Limitations:

- The dataset contains only 20 synthetic records and does not represent actual student outcomes.
- The relationship between study hours and marks is descriptive; it does not demonstrate causation.
- IQR-based outlier detection can flag legitimate observations and depends on the selected variables and threshold.
- The current workflow reads the supplied local CSV dataset; broader data-management features would require additional implementation.
- Fixed grade categories may not match every institution's assessment policy.

Possible future enhancements:

- Support user-uploaded CSV files with clearer validation feedback.
- Add semester-wise comparisons and historical progress tracking.
- Provide downloadable PDF summaries and configurable report options.
- Introduce a database for persistent student-record management.
- Improve dashboard accessibility and configurable grading policies.
- Explore predictive analysis only after collecting appropriate, sufficiently large, ethically managed data.

9. Conclusion

The Student Performance and Study-Hours Dashboard demonstrates an end-to-end Python workflow for loading structured data, validating records, encapsulating student information, computing statistics, comparing subject performance, detecting potential outliers, and presenting results through visualisations and a Streamlit interface. The 20-row example dataset supports demonstration and testing, not real-world inference. The project's main value is its integration of Python programming, object-oriented design, data analysis, visualisation, testing, and a browser-based dashboard in a software-only laboratory project.

10. References

20. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
21. NumPy Developers. NumPy Documentation. https://numpy.org/doc/
22. pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
23. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
24. Streamlit, Inc. Streamlit Documentation. https://docs.streamlit.io/
25. Project source code and synthetic dataset: Student Performance and Study-Hours Dashboard repository.

11. Appendix: Dataset Schema and Execution

CSV headers used by the supplied dataset:

student_id, study_hours, python_score, dsa_score, math_score, attendance_pct

student_id	study_hours	python_score	dsa_score	math_score	attendance_pct
S001	2.0	48	45	52	68
S002	3.5	62	58	65	76
S003	4.0	70	66	72	82

Execution guidance:

- Create and activate a Python virtual environment.
- Install dependencies with: python -m pip install -r requirements.txt
- Run the integrated analysis with: python main.py

- Launch the dashboard with: python -m streamlit run app.py
- Run the test scripts individually from the project folder to check core functionality.

Note: Use the local README.md for the current setup instructions. Exact execution behaviour depends on the version of the project source files included in the submitted repository.